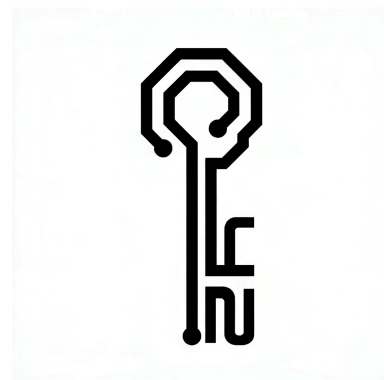


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 5
«Технология MPI. Введение»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Анализ алгоритма	3
3.1. Описание	3
4. Описание использованных функций MPI	5
4.1. MPI_Init	5
4.2. MPI_Comm_size	5
4.3. MPI_Comm_rank	5
4.4. MPI_Send / MPI_Recv	5
4.5. MPI_Barrier	5
4.6. MPI_Wtime	5
4.7. MPI_Reduce	6
4.8. MPI_Finalize	6
5. Разработанные программные коды	6
6. Результаты экспериментов	10
7. Заключение	14

1. Цель работы

Изучить базовые особенности применения технологии MPI.

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (_OPENMP = 202011).

3. Анализ алгоритма

3.1. Описание

1. На процессе с идентификатором `rank = 0` создается массив `array`, содержащий `count` элементов.
2. На процессе с `rank == 0` массив заполняется случайными числами, сгенерированными при помощи функции `rand()`.
3. Данные распределяются между процессами:
 - a. каждый процесс получает свою часть массива;
 - b. процесс с `rank == 0` передает фрагменты массива другим процессам с помощью вызова `MPI_Send`, а остальные процессы принимают эти данные посредством `MPI_Recv`.
4. Процессы параллельно определяют максимальное значение в своих подмассивах (локальный максимум).
5. Вычисляется глобальный максимум: с использованием функции `MPI_Reduce` локальные максимумы объединяются в одно глобальное значение на процессе с `rank == 0`.
6. Результаты выводятся на экран: процесс `rank == 0` выводит количество процессов, глобальный максимум и среднее время выполнения.

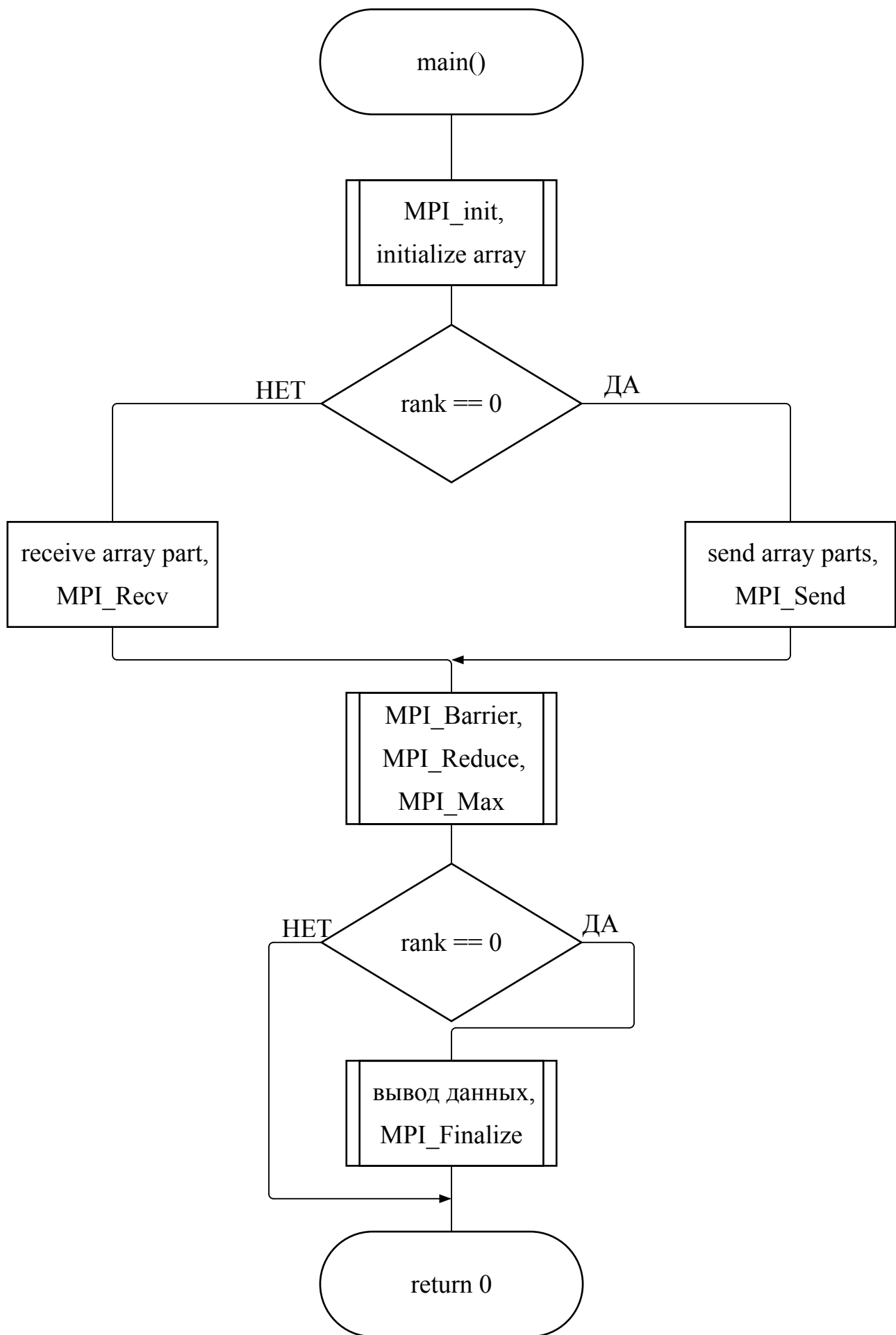


Рисунок 1 – Блок-схема алгоритма, описанного в п. 3.1

4. Описание использованных функций MPI

4.1. MPI_Init

- **Смысл:** запускает и подготавливает MPI-среду. Любое использование MPI-функций возможно только после этого вызова.
- **Роль в программе:** включает параллельное окружение и создаёт набор процессов для одновременного выполнения.

4.2. MPI_Comm_size

- **Смысл:** сообщает общее число процессов в заданном коммуникаторе (MPI_COMM_WORLD).
- **Роль в программе:** позволяет определить, на сколько частей делить данные при параллельной обработке.

4.3. MPI_Comm_rank

- **Смысл:** выдаёт номер (ранг) текущего процесса внутри коммуникатора.
- **Роль в программе:** помогает каждому процессу понять, какой участок задачи ему выполнять; ранг 0 играет роль управляющего процесса.

4.4. MPI_Send / MPI_Recv

- **Смысл:** механизмы отправки и получения сообщений между процессами.
- **Роль в программе:** процесс 0 рассылает фрагменты массива, остальные процессы принимают данные и обрабатывают их.

4.5. MPI_Barrier

- **Смысл:** останавливает выполнение каждого процесса, пока все остальные не достигнут этой точки.
- **Роль в программе:** обеспечивает корректное измерение времени параллельного участка.

4.6. MPI_Wtime

- **Смысл:** возвращает текущее время с высокой точностью.
- **Роль в программе:** используется для замера длительности вычислительной части программы.

4.7. MPI_Reduce

- **Смысл:** выполняет операцию объединения данных от всех процессов (например, MPI_MAX).
- **Роль в программе:** собирает локальные максимумы от всех процессов и определяет глобальный максимум на процессе с рангом 0.

4.8. MPI_Finalize

- **Смысл:** завершает работу MPI и освобождает связанные ресурсы.
- **Роль в программе:** корректно завершает параллельное выполнение после завершения вычислений.

5. Разработанные программные коды

```
#include <mpi.h>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <memory>
#include <vector>
#include <random>
#include <stdint>
#include <climits>
#include <limits>
#include <iomanip>
#include <cmath>

namespace fs = std::filesystem;

static const int repeats = 10;
static const int32_t count = 3*std::pow(10, 7);

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int size = 0;
    int rank = 0;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int base_count = count / size;
    int remainder = count % size;
    int local_count = base_count + (rank < remainder ? 1 : 0);

    std::random_device dev;
    std::mt19937 gen(dev());
    std::uniform_int_distribution<int> dist(0, 0x7FFFFFFF);

    double total_time = 0.0;

    for (int r = 0; r < repeats; ++r) {
```

```

std::unique_ptr<int[]> local_array = std::make_unique<int[]>(local_count);
int local_max = INT_MIN;
int global_max = INT_MIN;

if (rank == 0) {
    std::vector<int> global_array(static_cast<size_t>(count));
    for (int32_t i = 0; i < count; ++i) {
        global_array[i] = dist(gen);
    }

    for (int i = 0; i < local_count; ++i) local_array[i] = global_array[i];

    int32_t current_position = local_count;
    for (int dest = 1; dest < size; ++dest) {
        int dest_count = base_count + (dest < remainder ? 1 : 0);
        if (dest_count > 0) {
            MPI_Send(global_array.data() + current_position, dest_count,
                     MPI_INT, dest, 0, MPI_COMM_WORLD);
            current_position += dest_count;
        }
    }
} else {
    if (local_count > 0) {
        MPI_Recv(local_array.get(), local_count, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
}

MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();

for (int i = 0; i < local_count; ++i) {
    if (local_array[i] > local_max) local_max = local_array[i];
}

MPI_Reduce(&local_max, &global_max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);

double end = MPI_Wtime();

if (rank == 0) total_time += (end - start);
}

if (rank == 0) {
    double avg_seconds = total_time / static_cast<double>(repeats);
    double avg_ms = avg_seconds * 1000.0;
    std::string fname = "results/mpi_time";
    bool need_header_time = (!fs::exists(fname) || fs::file_size(fname) == 0);
    {
        std::ofstream file(fname, std::ios::app);
        if (!file) {
            std::cerr << "Cannot open " << fname << " for appending\n";
        } else {
            if (need_header_time) {
                file << 1 << "\n";
                file << count << "\n";
                file << "Кол-во процессов\n";
                file << "Время выполнения, мс\n";
            }
        }
    }
}

```

```

        file << "none\n";
    }
    file << size << " " << std::fixed << std::setprecision(6) << avg_ms << "\n";
}

double baseline_ms = std::numeric_limits<double>::quiet_NaN();
std::string baseline_fname = "results/mpi_baseline";
if (size == 1) {
    std::ofstream base(baseline_fname);
    if (base) {
        base << std::fixed << std::setprecision(6) << avg_ms << "\n";
        baseline_ms = avg_ms;
    } else {
        std::cerr << "Cannot write baseline file " << baseline_fname << "\n";
    }
} else {
    if (fs::exists(baseline_fname) && fs::file_size(baseline_fname) > 0) {
        std::ifstream base(baseline_fname);
        if (base) {
            base >> baseline_ms;
        }
    }
}

std::string sname = "results/mpi_speedup";
bool need_header_sp = (!fs::exists(sname) || fs::file_size(sname) == 0);
{
    std::ofstream sp(sname, std::ios::app);
    if (!sp) {
        std::cerr << "Cannot open " << sname << " for appending\n";
    } else {
        if (need_header_sp) {
            sp << 2 << "\n";
            sp << "Реальное ускорение\n";
            sp << "Ожидаемое ускорение\n";
            sp << "Кол-во процессов\n";
            sp << "Ускорение\n";
            sp << "none\n";
        }
        double realSpeedup = std::numeric_limits<double>::quiet_NaN();
        double theoreticalSpeedup = static_cast<double>(size);
        if (std::isfinite(baseline_ms) && avg_ms > 0.0) {
            realSpeedup = baseline_ms / avg_ms;
        }
        sp << size << " " << std::fixed << std::setprecision(6)
            << realSpeedup << " " << theoreticalSpeedup << "\n";
    }
}

std::string efname = "results/mpi_efficiency";
bool need_header_ef = (!fs::exists(efname) || fs::file_size(efname) == 0);
{
    std::ofstream ef(efname, std::ios::app);
    if (!ef) {
        std::cerr << "Cannot open " << efname << " for appending\n";
    } else {

```

```

        if (need_header_ef) {
            ef << 2 << "\n";
            ef << "Реальная эффективность\n";
            ef << "Ожидаемая эффективность\n";
            ef << "Кол-во процессов\n";
            ef << "Эффективность\n";
            ef << "none\n";
        }
        double realSpeedup = std::numeric_limits<double>::quiet_NaN();
        if (std::isfinite(baseline_ms) && avg_ms > 0.0) {
            realSpeedup = baseline_ms / avg_ms;
        }
        double realEfficiency = std::numeric_limits<double>::quiet_NaN();
        if (std::isfinite(realSpeedup)) realEfficiency = realSpeedup /
static_cast<double>(size);
        double theoreticalEfficiency = 1.0;
        ef << size << " " << std::fixed << std::setprecision(6)
            << realEfficiency << " " << theoreticalEfficiency << "\n";
    }
}

std::cout << "MPI measurement finished. processes=" << size
    << " avg_time=" << avg_ms << " ms\n";
if (std::isfinite(baseline_ms)) {
    std::cout << "Baseline T1 (ms) = " << baseline_ms << "\n";
} else {
    std::cout << "Baseline T1 not found. Run with 1 process first to create results/
mpi_baseline.\n";
}

std::cout << "Appended results to results/mpi_time, results/mpi_speedup, results/
mpi_efficiency\n";
}

MPI_Finalize();
return 0;
}

```

Файл 1 – Измерение времени для MPI реализации

6. Результаты экспериментов

Как и в первой лабораторной, каждая точка на графике была получена путем повторения эксперимента 10 раз, на одинаковых исходных данных для каждого повтора. Кол-во элементов: 30×10^6

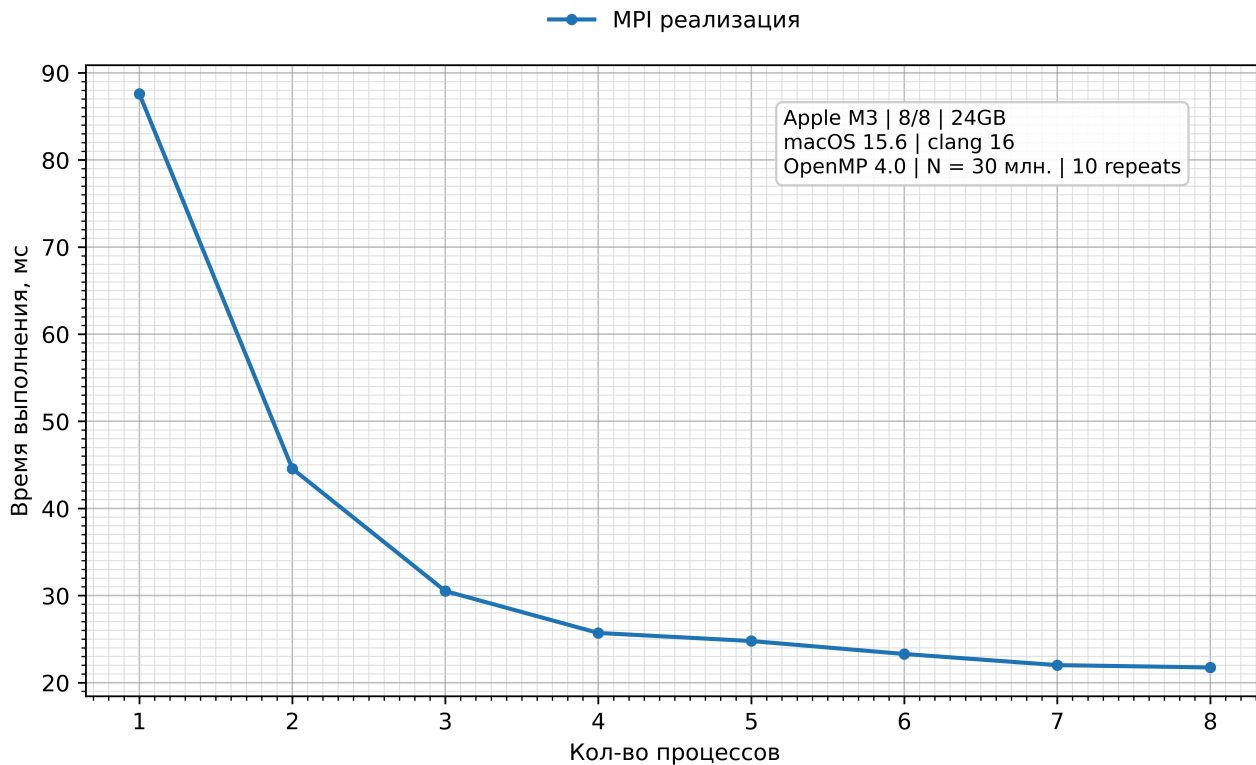


Рисунок 2 – График зависимости времени выполнения программы от количества потоков для MPI реализации

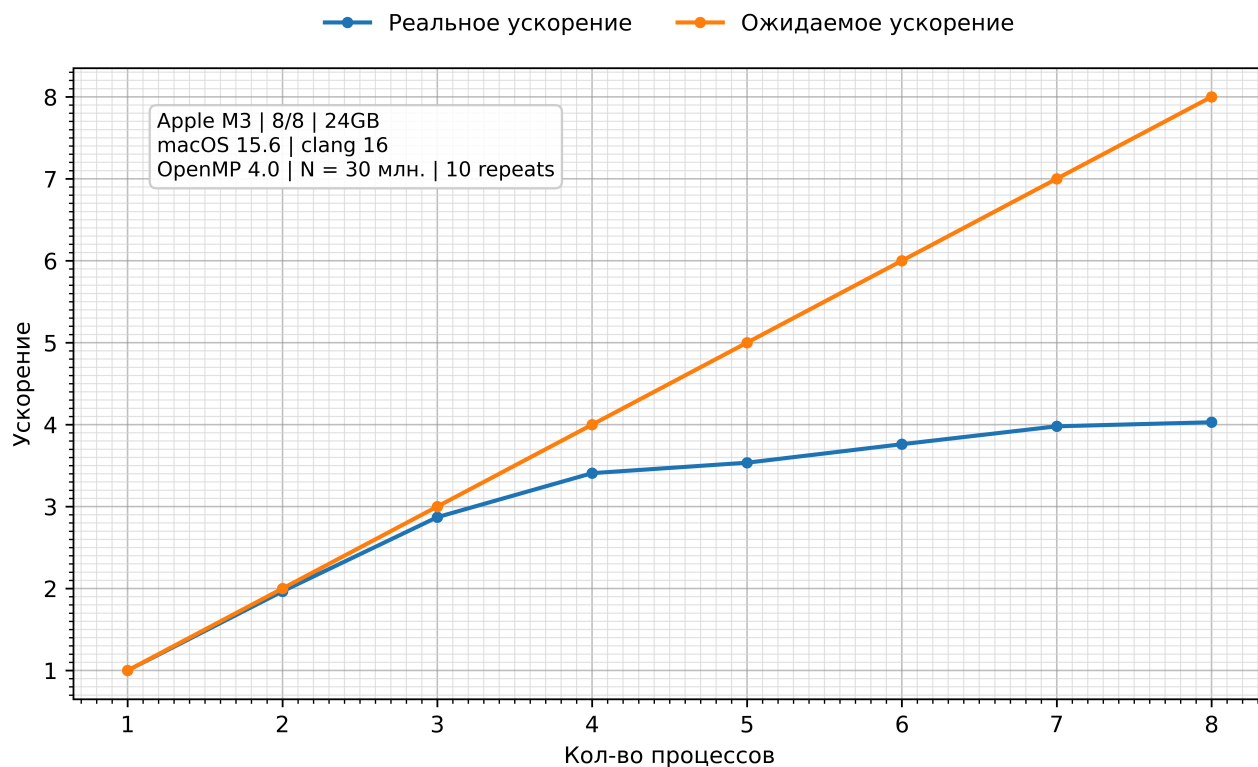


Рисунок 3 – График зависимости ускорения программы от количества потоков для MPI реализации

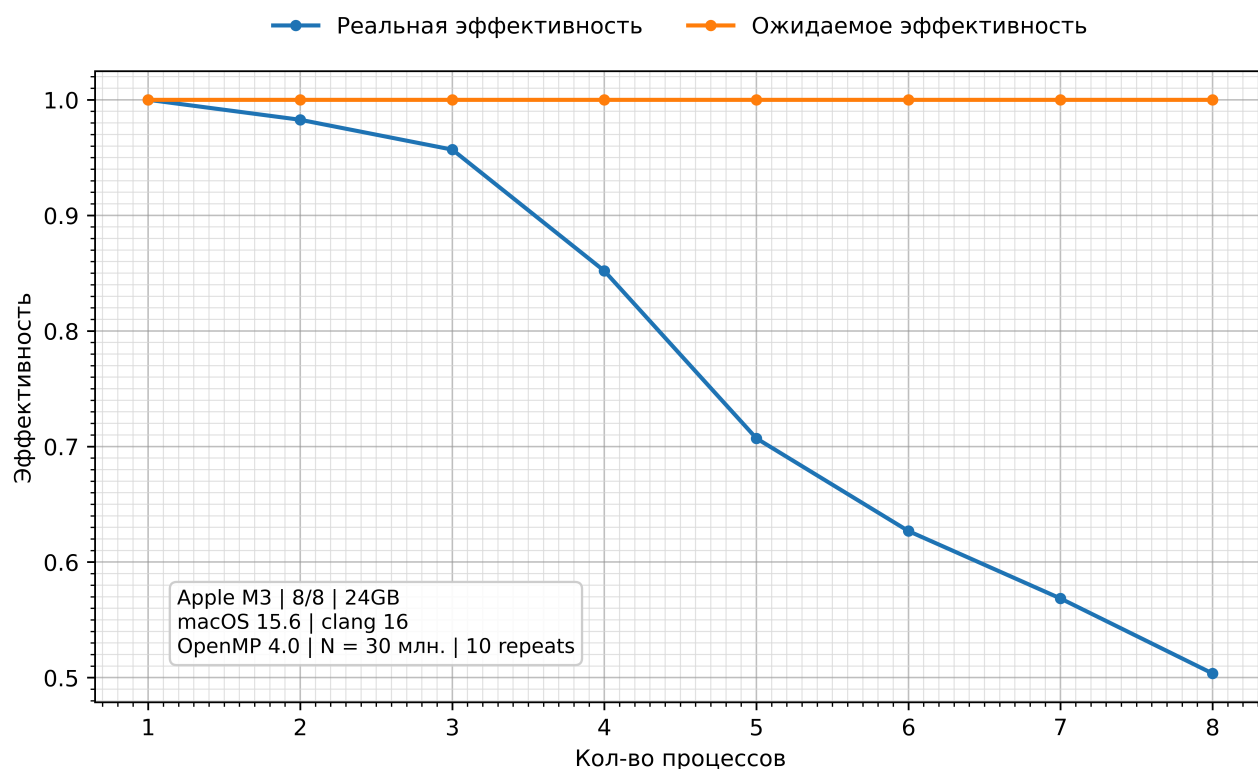


Рисунок 4 – График зависимости эффективности программы от количества потоков для MPI реализации

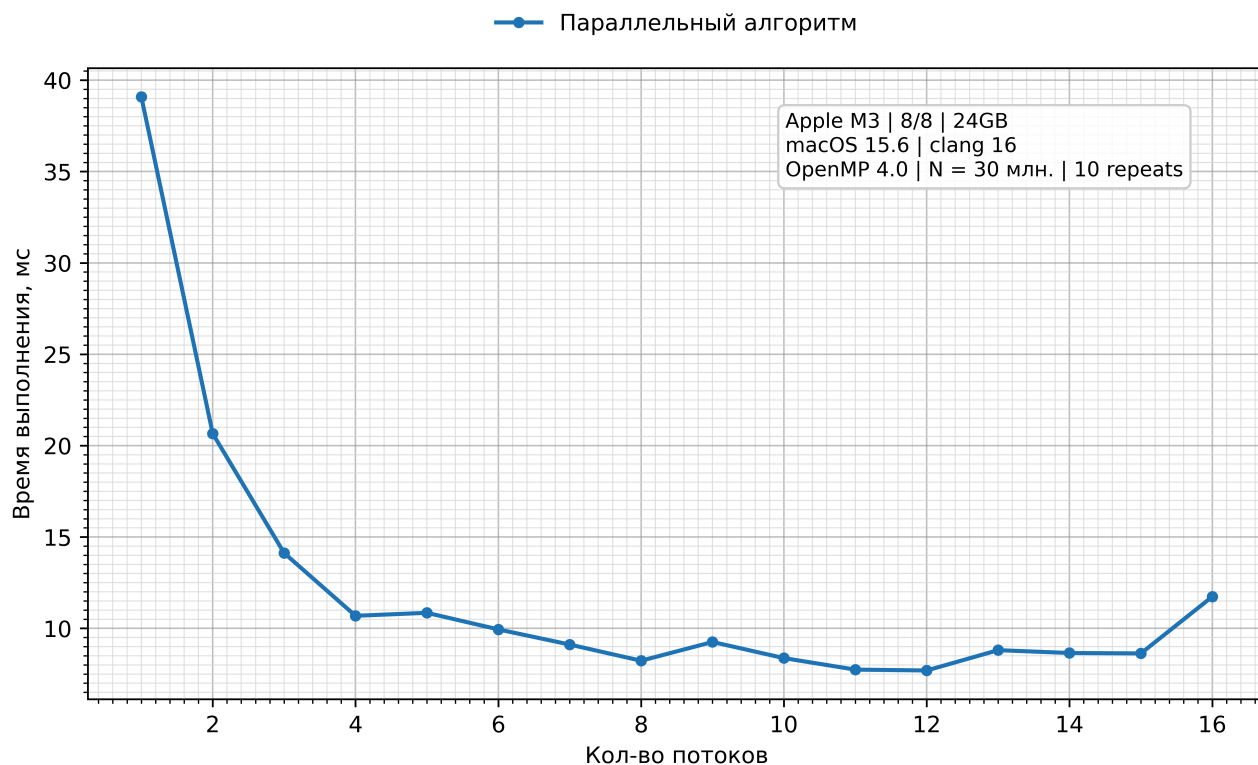


Рисунок 5 – График зависимости времени выполнения программы от количества потоков для OpenMP реализации из первой лабораторной

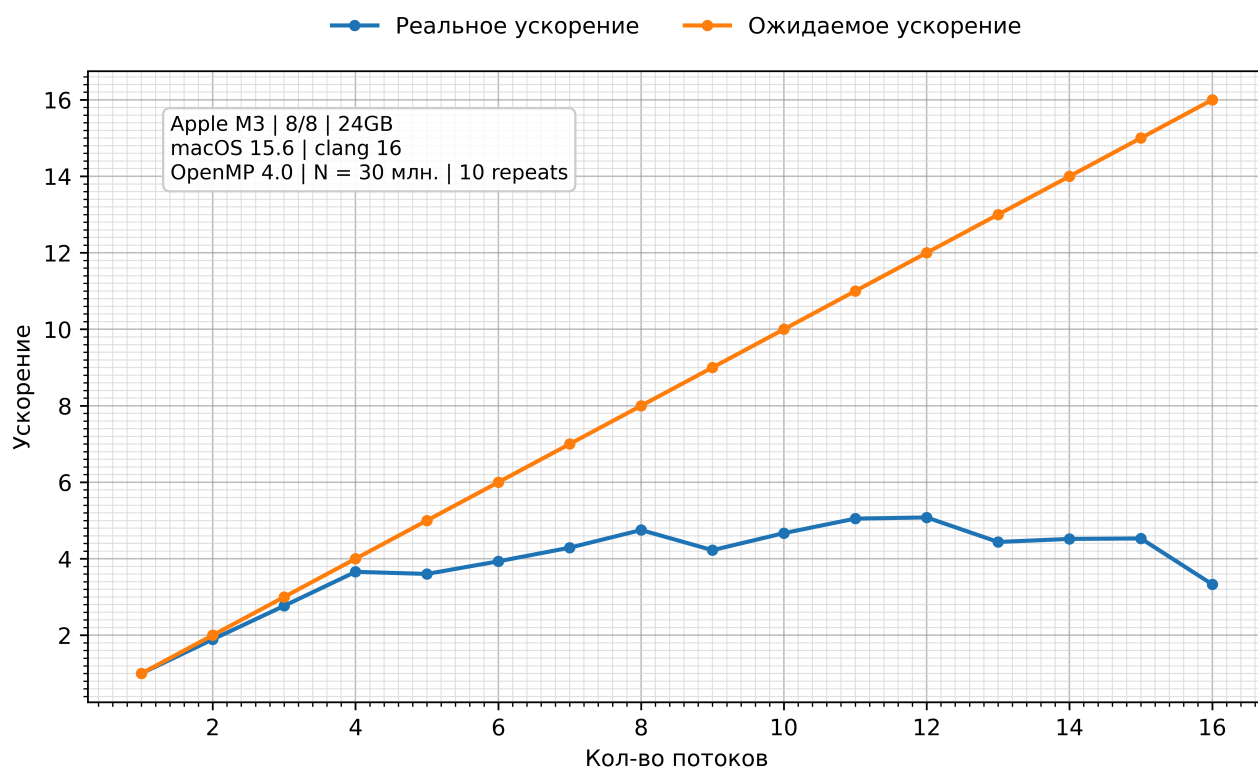


Рисунок 6 – График зависимости ускорения программы от количества потоков для OpenMP реализации из первой лабораторной

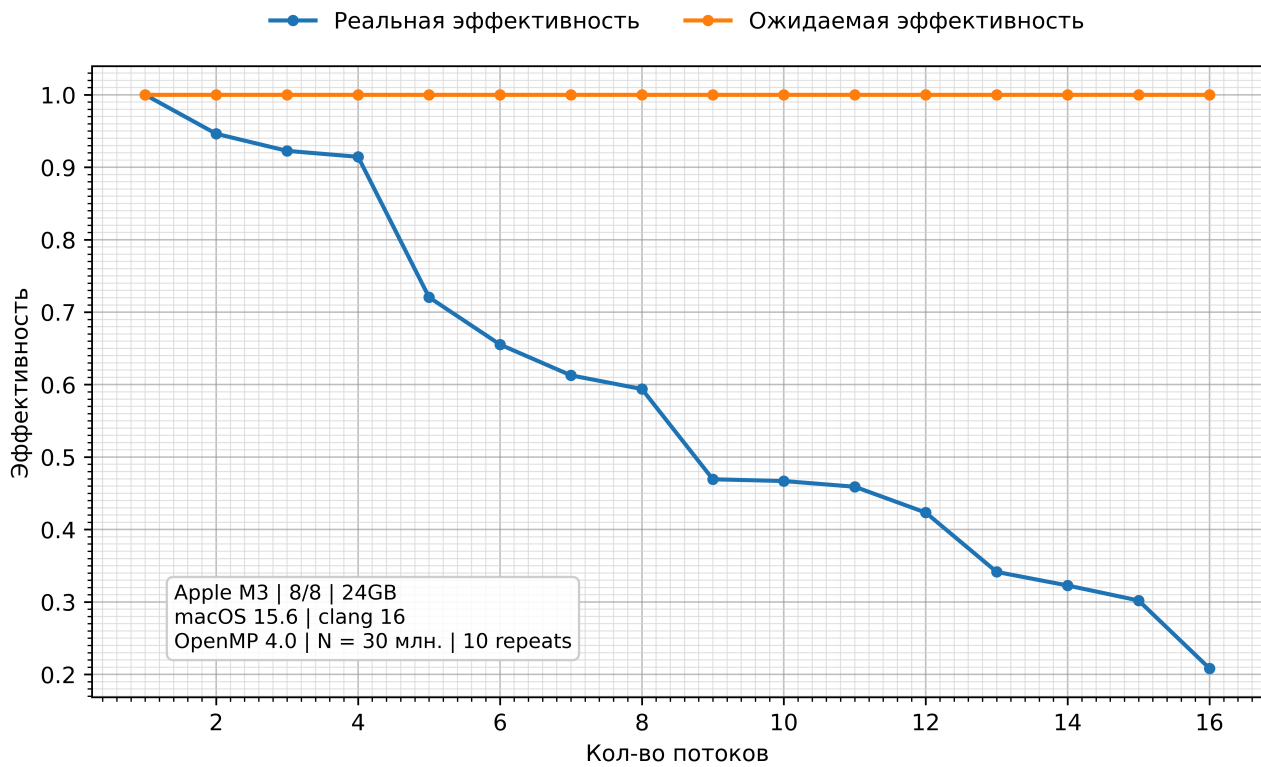


Рисунок 7 – График зависимости эффективности программы от количества потоков для OpenMP реализации из первой лабораторной

7. Заключение

В ходе выполнения лабораторной работы были рассмотрены основные принципы использования технологии MPI для организации параллельных вычислений. Была разработана программа, выполняющая поиск максимального элемента в массиве данных.

Сравнительный анализ MPI и OpenMP показал, что при запуске на одной вычислительной машине MPI обычно уступает OpenMP по производительности. Это связано с дополнительными затратами на обмен сообщениями между процессами, необходимостью вручную распределять данные и синхронизировать выполнение через коммуникации.

На графиках наблюдается прекращение роста ускорения и эффективности после достижения 4 потоков. Это связано с устройством процессоров Apple серии M: они используют 4 производительных и 4 энергоэффективных ядра. При 4 потоках достигается предел количества высокопроизводительных ядер, что ограничивает дальнейший линейный рост производительности (подробное объяснение см. в заключении лабораторной №1).

Полученные результаты подтверждают, что выбор между MPI и OpenMP зависит от специфики задачи и вычислительной системы. MPI может быть более эффективным на распределённых системах, но для параллельных вычислений на одной машине OpenMP часто даёт лучшие результаты из-за меньших накладных расходов на обмен данными между процессами.